TeleWire TRP-AO

Attention-Optimized Tag Recall Prompting for Stateful LLM Simulation

Attention-Optimized · FrankenSIM / Vexa · formerly FrankenWIRE

Method designer and project developer: Ryah

Whitepaper drafting, source review, and token analysis: Cindy, an LLM agent

Document: Technical design whitepaper, version 1.4 · November 3, 2026

Abstract

TeleWire TRP-AO (Attention-Optimized), previously documented as FrankenWIRE, is a symbolic prompt-authoring approach used in the FrankenSIM/Vexa interactive simulation project. It combines compact rule modules with distinctive Unicode and emoji cues, then references those rules through a staged page-and-function workflow. Its design objective is semantic density with explicit salience: preserve short instructions while spending a small token premium on recognizable markers for operations, relationships, and control flow.

Base TeleWire Tag Recall Prompting (TRP) prioritizes token compactness. A reproducible comparison of 17 AO samples against reconstructed compact base-TRP counterparts measured **12.94–13.82% AO overhead** across the selected mix, and **9.72–10.17%** across its twelve rule modules. Against ordinary Markdown prose versions of sixteen instruction/setup samples, AO measured **15.12–15.69% savings**, and compact base TRP **24.40–24.42%**. Three separate expression-level examples measured **39–57% savings against their English expansions**. These counts quantify the representation tradeoff; the behavioral value of the markers is a separate evaluation question.

Ryah also uses **targeted semantic loading** in development: choosing recurring state markers to reinforce the intended interpretation of an overloaded concept. The Ludus/🎲 case documents the observed problem, intervention, and intended behavior. This paper presents that development account alongside source inspection and scripted token measurements.

Authorship and verification process

Ryah designed the TRP-AO adaptation, developed the FrankenSIM/Vexa implementation, and supplied its design rationale and development observations. Cindy, an LLM agent, drafted and revised this whitepaper, inspected the supplied source, constructed comparison fixtures, and ran the token-counting scripts. References to Ryah identify the method's designer and developer; the document's explanatory voice and analysis were written by Cindy.

The verification process distinguishes **source inspection**, **scripted measurements**, **Ryah's development observations**, and **proposed behavioral tests**. Numerical results come from deterministic tokenizer runs on published paired strings, rather than an LLM's estimate of token cost. The base reconstructions and prose expansions were authored by Cindy and remain inspectable, so readers can assess the comparison choices and reproduce the counts.

This division makes the drafting and checking process explicit. Its evidentiary value comes from traceable sources, disclosed comparison choices, and reproducible calculations. LLM involvement alone does not establish reduced bias or independent validation: fixture selection and interpretation still involve judgment, and the work was performed within Ryah's project workflow.

---

1. Problem and design goals

A stateful language-model simulation must coordinate several concerns within each response: what participants know, which actions are possible, how events progress, how characters speak, and which facts survive into the next turn. When these concerns are interleaved in a long instruction document, a small edit can obscure a condition, create a competing rule, or change an output field that another component expects.

TRP-AO addresses both representation efficiency and specification maintenance. Rules need identifiable homes; procedures need explicit entry points; serialized state needs stable fields. The design targets five practical outcomes:

- **Traceability:** connect a behavior to its rule module and the procedure that checks it.
- **Salient semantic cues:** use compact, distinctive markers to signal instruction roles and recall established operations.
- **Local edits:** revise a concern without unnecessarily repeating or rewriting other concerns.
- **Continuity:** carry established simulation facts forward through a defined state envelope.
- **Reviewability:** expose reference drift, conflicting constraints, and missing route exits during inspection.

Compactness supports these goals when it preserves meaning. Token count is one objective; distinguishable instruction roles and stable interpretation are others.

---

2. Design lineage and the Vexa adaptation

The supplied TeleWire TRP guide proposes short tags, key-value properties, substitution arrows, and explicit recall of named rules. Its compactness objective favors segmentation that avoids spending multiple tokens where a compact representation is available. Ryah's TRP-AO adaptation retains that organization but deliberately broadens the symbolic vocabulary. The earlier working name FrankenWIRE is retained in existing artifact paths for compatibility.

Vexa uses pipes, Unicode arrows, uppercase stage labels, underscore names, comparison operators, and numbered procedures. These express relationships and mechanical conditions that would be lost in a universal conversion to flat properties and replacement arrows. Their presence reflects an intentional optimization

tradeoff, rather than merely departures from a formatting guide.

APPROACH	PRIMARY REPRESENTATION OBJECTIVE	TRADEOFF
Base TeleWire TRP	Minimize token cost while retaining the intended instruction	Favor economical segmentation and a restricted notation
TeleWire TRP-AO	Retain compactness while making instruction roles more distinct	Accept some additional marker cost for semantic cues and explicit structure

Ryah's earlier 5–10% estimate describes AO overhead against compact base TRP; the reported 30–50% savings uses expanded wording as its denominator. The reconstructed comparison in section 8 measures about 10% overhead for rule modules and 13–14% for the selected mixed pack. These results refine the earlier estimate without treating it as a universal bound.

ELEMENT	VEXA INTERPRETATION
`⇥ <page:...>`	Transition to another stage
`<knowledge_sanity>`	Named module defining knowledge constraints
`<state:...>`	Named part of the persistent output structure
`➤ <fn:...>` / `⇤`	Procedure invocation and return
`clause | clause`	Compact separation of related clauses or alternatives
`condition → consequence`	Implication or substitution, interpreted in context
`IF ct%3=0`	A conditional turn-based instruction
`scope: spoken dialogue only`	Explicit applicability of a rule

These symbols form an authoring convention. Their meanings are specified by the prompt and its host integration rather than a formal grammar enforced by a standalone compiler. Existing names and scopes take precedence over cosmetic normalization.

---

3. Architecture

The design separates three concerns within a prompt specification:

1. **Declarative modules** describe standing rules and constraints.
2. **Procedural pages and functions** organize which decisions and checks are requested for the current turn.
3. **State and output contracts** describe what must be emitted and retained.

The host surrounds this specification with character data, conversation history, macro values, and rendering behavior. A useful conceptual view is:

```
Rules + character data + prior state + user input + host values
                                ↓
                    Model-guided page route
                                ↓
                Response + serialized updated state
                                ↓
            Host rendering and subsequent prompt assembly
```

The diagram shows the prompt and host data flow. Page routing is interpreted by the model; the host handles context assembly and rendering.

Declarative modules

The inspected preset uses `<sys>` for general instructions, `<knowledge_sanity>` and `<epistemic_contract>` for information boundaries, `<npc_voice>` for speech, `<prose>` for narration style, and `<format>` for output targets. Other modules define world events, relationships, and spatial state.

Grouping rules by concern provides a named location for revision. It also allows procedures to refer back to a rule's scope instead of reproducing its full wording each time. Module naming is preserved exactly when another prompt section, macro, or renderer depends on it.

Procedural routing

The normal Antithesis route observed in Vexa Alpha v5 is:

```
boot → mechanics → router → solo OR ensemble → tail → aux → output
```

STAGE	INTENDED RESPONSIBILITY
`aux`	Accommodate additional decisions not covered earlier
`boot`	Establish turn context, scene flags, and continuity requirements
`mechanics`	Process narrative systems, participant turns, and optional world events
`output`	Produce the response and required state envelope
`router`	Apply conditional calls and select a scene-processing route
`solo` / `ensemble`	Process spotlight participants and their interaction targets
`tail`	Apply style, knowledge, continuity, and presentation checks

The router distinguishes the scene paths by spotlight count: at most one participant selects `solo`; two or more selects `ensemble`. Functions supply reusable procedures within this route. Their notation borrows familiar concepts such as calls, parameters, loops, and returns, while remaining instructions interpreted by the model.

---

4. Tag recall and attention-oriented markers

TRP connects a named rule module to a decision or check later in the prompt. In this design, *recall* means referring the model back to an available tagged specification. It does not describe vector retrieval, external search, or a separate memory database.

For example, the knowledge module asks that a participant's knowledge have an information vector: observation, a message, testimony, evidence, or common knowledge. A later procedure can check whether a proposed action relies on a fact that actually reached that participant.

The following is a simplified explanatory example, not a verbatim extraction or a drop-in replacement for the preset:

```
<knowledge_sanity>
scope: participant knowledge
sources: observation | message | testimony | evidence | common knowledge
no source → unaware of that fact
</knowledge_sanity>

<fn:knowledge_check target="X">
1. Check each action-relevant fact against <knowledge_sanity>.
2. Remove assumed knowledge without a supported source.
⇤
</fn:knowledge_check>
```

Semantically loaded cues

A marker can recall a definition much longer than its surface representation. Once the notation is established, `∀` indicates application to each relevant item; `∅` represents a null result or skipped effect; `➤` signals a procedure call; `⇥` signals a page transition. An emoji can identify a relationship axis without spelling out the axis name on every state entry. The intended compression comes from reusing a stable meaning, not from a symbol creating meaning without a definition.

Ryah's attention-oriented hypothesis is that uncommon, semantically loaded markers stand out from surrounding prose and help the model distinguish instruction types and recall the associated rules. Consistent marker roles and explicit recall are central to that design. Definitions, legends, and invocation context still consume tokens and belong in whole-prompt accounting.

The term *Attention-Optimized* names this design objective. Rarity alone does not establish that a model must attend to a marker or comply with its meaning. In the standard Transformer formulation, attention weights depend on query/key compatibility; there is no general rarity-priority rule [6]. Controlled comparisons can distinguish effects of marker choice from changes in surrounding instructions for a particular model.

Targeted semantic loading: the Ludus/🎲 development case

Ryah uses **targeted semantic loading** to describe deliberate contextual framing through the choice and repetition of a symbol. The target is the interpretation associated with a field: which part of an overloaded concept should remain active as state is repeatedly emitted and read.

The Aether Matrix uses twelve relationship axes—six fondness axes and six friction axes—to represent relationships extending beyond romance. Its names draw on love-related vocabulary, but the matrix and its combinations are project-specific; they are not a standardized twelve-axis ancient theory. Related modern love-style terminology appears in John Alan Lee's typology [8]. Here, **Ludus is intended to act as a playfulness modifier** whose expression depends on the other axes:

INTENDED COMBINATION	CONTEXTUAL EXPRESSION
High Eros + high Ludus	Playful romantic attraction
High Philia + high Ludus	Close friendship expressed through joking and banter
High Storge + high Ludus	Playful familial interaction, such as sibling teasing

These are intended examples, not exhaustive or deterministic relationship classifications. The full matrix, established relationship, and character context still govern expression.

During development, Ryah observed Ludus drifting into a dominant flirting signal across otherwise different relationships. Ryah attributed this to the romantic framing of the love vocabulary and romance/drama tendencies in the fictional scenarios being tested. The observed problem was semantic overreach: the label's romantic associations displaced its intended modifier role. Section 9 proposes a controlled comparison of that diagnosis and intervention.

The intervention changed the definition and recurring representation. 🎲 **binds Ludus to game/play in the initial legend; subsequent Internal States use the emoji without repeating the word.** Cindy confirmed that the inspected enabled prompt contains “Ludus” once, in the axis legend, followed by `Emit emoji only. Never axis names.` Its pair-state template stores the axis as `🎲[v,a]`.

The intended feedback path is:

```
Define Ludus with 🎲 as game/play
                  ↓
Carry 🎲, without the word, in recurring relationship state
                  ↓
Repeated play-oriented framing when state returns to context
                  ↓
Express playfulness through the other active relationship axes
```

Ryah describes this as repetition bias and semantic priming: reinforce the desired association while reducing repeated exposure to the overloaded label. Emoji choice therefore carries a semantic purpose as well as a compactness or salience purpose. Replacing 🎲 with a romantic symbol, an arbitrary marker, or the word

“Ludus” everywhere would change that intervention even if the numeric values stayed identical. This concerns conditioning within context, not a claim of model-weight training.

Ryah actively uses and tests this strategy during development. Cindy's source inspection confirms the implementation. The behavioral evidence available for this paper is Ryah's qualitative account; a quantified effect size requires matched trials and recorded outputs.

---

5. Scope, notation, and semantic preservation

The dialect permits concise rules without making every rule use the same syntax. Substitution is useful when a desired alternative can be stated clearly. An explicit prohibition remains useful when an exclusion itself is the required boundary.

Scope is especially important. In the inspected preset, `<npc_voice>` governs spoken dialogue, while `<banned_vocabulary>` applies to both speech and narration. Merging those blocks or dropping the scope statement could change behavior even if the resulting prompt were shorter.

Symbolic differences also carry meaning. `A→B` can express direction; `A↔B` can describe a pair; `⇥ <page:output>` describes a route transition. A style rewrite must preserve those distinctions. The same applies to macro names, state keys, function parameters, units, ranges, and exception conditions.

A practical review question is: **Can the revised specification recover the same actor, condition, action, scope, and retained state?** If the answer is uncertain, the edit requires more than a punctuation cleanup.

---

6. Persistent state and host boundaries

Vexa requests a terminal `<internal_states>` structure containing records for the world, locations, participants, relationships, narrative tracks, and other ongoing concerns. Within the simulation, the design treats that structure as the continuity record for established world facts.

Spatial records illustrate the value of a stable contract. Room identifiers, coordinates, floor membership, and adjacency links must stay consistent across turns if a map renderer is to represent the same place. Renaming a field or reconstructing geometry during an unrelated prose edit can change the visible result.

The design therefore distinguishes two kinds of control:

CONTROL	RESPONSIBILITY AND LIMIT
Host-level integration	Resolves supported macros, assembles context, and applies configured parsing/rendering rules; exact behavior depends on the installed host
Prompt-level instruction	Requests decisions, sequencing, and output; execution remains model-dependent

An instruction to preserve state does not itself provide durable storage. Continuity depends on emitted records being retained, included in later context, and interpreted consistently. Likewise, declaring a procedure or extension slot does not prove that a host converter or consumer is connected.

---

7. A turn-level walkthrough

Consider a user entering a room where two established participants are already present.

1. **Context:** boot establishes the current location, turn, and scene flags using the available conversation and state.
2. **World progression:** mechanics requests participant turns and the optional world-event pass. Vexa explicitly places the latter after ordinary world turns and gives it a single-event budget.
3. **Routing:** the spotlight count selects the ensemble path.
4. **Participant decisions:** procedures refer to identity, goals, knowledge, and relationship-related state for each relevant participant and target.
5. **Checks:** tail requests continuity, knowledge, prose, and presentation checks before output.
6. **Serialization:** the response is produced with the required header and updated state records for subsequent turns.

This walkthrough illustrates the route specified in the preset. Section 9 defines how to test its observable outputs and state updates.

---

8. Implementation evidence and token measurements

This paper is grounded in the preset snapshot `[Lumi] Vexa - Alpha v5.json`, inspected on November 3, 2026. The file contains **63 prompt blocks**, of which **58 are enabled** and **40 are enabled with nonempty content**. Empty or marker blocks are included in the first two counts; these numbers describe configuration structure, not runtime work or performance.

Inspection supports the presence of tagged rule modules, symbolic guards, parameterized procedures, the documented route, and a serialized state template. The project also contains a reusable agent skill documenting the modified dialect and its differences from the supplied guide.

Cindy also identified unresolved reference names, competing dialogue-ratio instructions, and a malformed function closing delimiter. These are maintenance findings in the inspected snapshot. They remain relevant to runtime acceptance and do not affect the deterministic counts of the frozen strings below.

Ryah's development estimates

Ryah reports repeated savings typically around **30–50% relative to expanded prompt wording**, and initially estimated **5–10% overhead relative to compact base TRP**. These are development estimates. Cindy measured the comparisons below from explicit fixtures to give the paper reproducible numerical

evidence.

Measured AO overhead against reconstructed compact base TRP

Cindy froze **17 source-derived AO samples** before tokenization and authored compact base-style counterparts using the supplied TRP guide: twelve complete rule modules, three procedural excerpts, one axis legend, and one matrix state row. The AO strings match the inspected snapshot after newline normalization. Counts use `tiktoken` 0.14.0 with two explicit reference encodings [10].

SCOPE	ENCODING	RECONSTRUCTED BASE	AO	AO OVERHEAD
All 17 samples, once each	`cl100k_base`	2,098	2,388	+13.82%
All 17 samples, once each	`o200k_base`	2,086	2,356	+12.94%
Twelve rule modules	`cl100k_base`	1,691	1,863	+10.17%
Twelve rule modules	`o200k_base`	1,688	1,852	+9.72%
Three procedural excerpts	`cl100k_base`	290	344	+18.62%
Three procedural excerpts	`o200k_base`	283	333	+17.67%

AO overhead is `100 × (AO / base − 1)`, calculated from each group's strings joined with two newlines. **The measured estimate is about 10% for these rule modules and 13–14% for this mixed selection.** The range reflects the two encodings, not a statistical confidence interval. The base texts are reconstructions, and the selection is a stratified convenience sample. Intended meaning was reviewed; behavioral equivalence was not tested. These are exact counts for the published strings and a sample-based estimate of AO cost, rather than a whole-preset overhead measurement.

An additional control replaced Unicode markers with ASCII equivalents while retaining the AO wording. Across the twelve rule modules, AO cost only **0.22% more** under either encoding. Most of the roughly 10% AO-versus-reconstruction gap in those modules therefore comes from changes to tags, wording, and format, rather than the net cost of Unicode markers alone.

The isolated matrix row adds **35–39 tokens** over short axis labels, or **66–74%** on that small row. In a separate denominator check, replacing only the legend and row labels inside the otherwise unchanged full state template gives **1.66–1.89%** overhead. Neither percentage measures a live conversation: actual state cost depends on emitted records and retained history. Short labels also change the targeted semantic-loading intervention.

The comparison report, all paired strings, and recorded results expose the reconstruction choices and diagnostics. Run `python docs/frankenwire/trp-base-comparison/compare.py` from the repository root to reproduce them.

Ordinary Markdown instruction baseline

Cindy added ordinary prose versions of the same samples using Markdown headings, bullets, numbered steps, and full sentences. The baseline was written and frozen before its token counts were run, without a target savings rate. It was added after the original AO/base comparison. The original source excerpts and compact-base reconstructions are unchanged.

INSTRUCTION/SETUP SCOPE	ENCODING	MARKDOWN	COMPACT BASE	AO	BASE SAVINGS	AO SAVINGS
All sixteen	`cl100k_base`	2,705	2,045	2,296	24.40%	15.12%
All sixteen	`o200k_base`	2,690	2,033	2,268	24.42%	15.69%
Twelve rule modules	`cl100k_base`	2,229	1,691	1,863	24.14%	16.42%
Twelve rule modules	`o200k_base`	2,225	1,688	1,852	24.13%	16.76%
Three procedures	`cl100k_base`	346	290	344	16.18%	0.58%
Three procedures	`o200k_base`	341	283	333	17.01%	2.35%

Across these sixteen instruction/setup samples, AO saves about 15–16% against concise Markdown instructions; compact base TRP saves about 24%. Savings use `100 × (1 − compact / Markdown)`. Headings and Markdown punctuation are included. These are counts for one authored prose rendering, so the result depends on wording and does not measure how all instruction writers compose prompts. Required identifiers, examples, conditions, and output syntax are retained; no model-generation equivalence test was run.

The serialized matrix row is not a prose instruction and is excluded from this primary comparison. The separate seventeen-sample mixed-pack diagnostic retains the exact AO state row in the prose baseline as a shared output contract; base retains its earlier short-label row. The prose axis legend retains the required emoji mappings. Full strings and mixed-pack totals are available in [10].

The smaller savings here than in the three expression examples below reflect a different baseline and sample: concise, complete Markdown instructions versus selected expanded expressions. The measurements describe their respective strings; the expression examples do not establish a general 39–57% reduction for full instruction modules.

Measured illustrative savings against expanded prose

Cindy paired three compact expressions with explicit English expansions and counted them with `tiktoken` 0.14.0. The table reports measured savings for those examples. Both encodings are reference counters; production-model tokenization and billed usage are outside this measurement.

EXPRESSION	`CL100K_BASE`: PROSE → AO	SAVED	`O200K_BASE`: PROSE → AO	SAVED
`AGE ∀ g +1`	11 → 5	54.5%	11 → 6	45.5%
`IF ct%3=0 ➤ <fn:chekhov_fire>`	28 → 17	39.3%	27 → 16	40.7%
`n ∉ K(g) → add n`	21 → 9	57.1%	21 → 9	57.1%

The exact English expansions were:

1. “Increase the age of every gossip record g by one.”
2. “If the current turn count ct is divisible by three, call chekhov_fire; after the function returns, continue with the next instruction.”
3. “If participant n does not belong to the gossip witness set K(g), add participant n to that set.”

These comparisons assume the meanings of `g`, `ct`, `K(g)`, and the call/return notation are already established. They exclude definition/legend costs, prompt wrappers, macro expansion, output tokens, and provider formatting. Intended semantic correspondence was reviewed for illustration; no generation-based equivalence test was run. The full strings and results are recorded in `token-evidence.json`; `token-evidence.py` reproduces the counts [5].

Marker cost and measurement boundaries

Many useful markers cost one to three tokens in the reference encodings, but glyph count and token count are different quantities. Some compound symbols and emojis cost more, and preceding spaces can change segmentation [7]. Selected standalone counts are:

MARKER	`CL100K_BASE`	`O200K_BASE`
`→`	1	1
`⇥`	3	2
`∀`	2	1
`➤`	2	2
🌶️	4	3
𝒻	4	4

Shorter text can produce real token savings, as the expression counts show. Smaller character count by itself is not proof of fewer tokens: vocabulary, bytes, and context determine segmentation. This distinction preserves the token-efficiency claim while making its measurement reproducible.

The measurements establish token cost for the disclosed strings. Ryah's Ludus account supplies qualitative behavioral evidence, and source inspection establishes how the intervention is encoded. Controlled measurements of adherence, state accuracy, quality, and latency remain the next evaluation step.

---

9. Evaluation plan

A useful evaluation should compare observable outcomes under a fixed model/version, host configuration, sampler settings, character data, and starting state. First resolve known contradictory instructions so that a failure has an unambiguous expected result.

Compare expanded prose, compact base TRP, and TRP-AO under equivalent task and rule semantics. Add ablations that replace AO symbols with ASCII/word equivalents and remove explicit recall while keeping the rule content. That separates compression from marker selection and route/recall effects. Treat changes in meaning as a confound rather than a compression gain.

QUESTION	PROPOSED MEASUREMENT
Are knowledge boundaries respected?	Unsupported knowledge events in predefined scenes
Are requested constraints followed?	Passed applicable constraints divided by all applicable constraints
Do AO markers improve observable adherence?	Same-task compliance and state scores with marker/recall ablations; rarity alone is not the outcome measure
Does routing affect observable output?	Expected route-dependent behavior across isolated, multi-participant, and mode-switch cases
Does state remain consistent?	Valid retained/updated fields divided by fields checked against the fixture
Does targeted semantic loading preserve Ludus as a modifier?	Unwarranted flirting in friendship/familial scenes at fixed relationship values, alongside retention of intended playfulness and appropriate romantic expression when Eros/context support it
Is there an efficiency tradeoff?	Assembled-input tokens, output tokens, latency, and estimated cost under the same configuration
Is the specification easier to maintain?	Time and errors for equivalent scoped edits, including reference repair

Use repeated trials and publish trial counts and variation. For subjective quality, use blinded scoring criteria and keep it separate from mechanical correctness. A model-written explanation of following the route is not an execution trace; assess visible outputs and state, with host instrumentation where available.

For the Ludus case, hold the numeric matrix, persona, relationship, scene, and model configuration fixed while comparing repeated word labels, word-plus-emoji labels, and emoji-only state after the same definition. Add a token-budget-matched arbitrary-symbol control where feasible. Assess whether outcomes depend on the play-associated symbol, removal of the repeated word, or their combination; do not treat ordinary sibling/friend playfulness as evidence of flirting.

The token comparisons in section 8 are complete and reproducible. The controlled behavioral evaluation described in this section remains proposed; it would extend the available source evidence and development observations with matched output trials.

---

10. Engineering contribution

The work demonstrates an applied prompt-design tradeoff: retain compact symbolic instructions while allocating additional tokens to distinctive semantic cues and explicit procedural structure. It combines tagged rule recall with Vexa's page/function workflow and persistent simulation contracts, while preserving scope and host interfaces.

Relevant engineering work includes prompt architecture, context organization, interface maintenance, state modeling, and test design. The paper makes no claim to invent XML tagging, symbolic notation, state serialization, or modular procedures. Its contribution lies in their project-specific composition and the accompanying implementation documentation.

Targeted semantic loading extends that contribution to representation choices inside persistent state: select a symbol for the association it should reinforce, then preserve its binding across repeated context. The Ludus example documents Ryah's development process and motivates a specific, testable framing comparison.

References and reproducibility

[1] Implementation snapshot. Ryah, *[Lumi] Vexa — Alpha v5*, inspected November 3, 2026. Source artifact: `New folder/[Lumi] Vexa - Alpha v5.json`. SHA-256: `a26276225f8beccd06bfc0cddde916d485541c2cd0607614c13a0547fa2dbd88`.

[2] Dialect comparison. *Observed FrankenSIM / Vexa dialect*, project reference dated November 3, 2026: `.agents/skills/frankensim-frankenwire-syntax/references/vexa-v5-dialect.md`. Records syntax evidence and unresolved issues by block name.

[3] Design background. *The FrankenWIRE / TeleWire TRP Guide*, user-supplied at the FrankenSIM GitHub Wiki, retrieved November 1, 2026. The archived UTF-8 document has SHA-256 `e2c2bbd1b1dff35e8a78c928de32de5435309fbf97d77edcd809f40b087b7ef2`. TRP-AO adapts its compactness-oriented grammar; this revision distinguishes reproduced token counts, Ryah's development observations, and proposed behavioral tests.

[4] Author clarification and notation. On November 3, 2026, Ryah named the method TeleWire TRP-AO, explained its compactness/salience tradeoff, and reported the token ranges above. The supplied NOTATION.md, document revision `c223e8c7a6539e57a063fde135412d0dca40bfe3`, provides

mathematical/operator meanings. Snapshot SHA-256: `fb5bc1ab50309e216292f393aac0d424df4d0950c3f720b84059bf5a43450d7a`. It predates portions of the current boot protocol and Antithesis library; current preset definitions take precedence. For example, its two definitions of the all-axis set `Ψ` differ on inclusion of self-axis `ϕ`; that glossary inconsistency is not a new rule for the preset.

[5] Reproduced token examples. `token-evidence.py` and `token-evidence.json`, distributed alongside this paper. Run `python token-evidence.py` with `tiktoken` installed; the recorded version is 0.14.0. Cindy authored the English expansions for this comparison; they are separate from Ryah's historical development examples. No model generation is performed.

[6] Attention formulation. Vaswani et al., *Attention Is All You Need*, section 3.2. Cited to explain attention-weight computation, not as evidence validating TRP-AO.

[7] Tokenization background. Hugging Face, *Tokenization algorithms*. Explains vocabulary-dependent subword and byte-level segmentation; this is why character savings and token savings must be distinguished.

[8] Love-style terminology. John Alan Lee, *A Typology of Styles of Loving*, *Personality and Social Psychology Bulletin*, 3(2), 173–182 (1977); the article also cites Lee's *Colours of Love* (1973). This is related terminology, not a claim that Lee's six styles define the project's twelve independent axes or its intended Ludus combinations.

[9] Targeted semantic loading development account. Ryah's explanation supplied November 3, 2026: the observed Ludus-to-flirting drift, the intended contextual combinations, the definition/emoji intervention, and active development use. Source inspection confirms the one-occurrence legend and emoji-only state instruction in implementation snapshot [1]. Cindy inspected the implementation; the behavioral account is supplied by Ryah.

[10] Reconstructed compact-base comparison. Cindy, November 3, 2026. Report, frozen fixtures, results, and counting script. Seventeen primary pairs frozen before counting, with separate ASCII-marker and state-template diagnostics; `tiktoken` 0.14.0, `cl100k_base` and `o200k_base`. Fixture SHA-256: `a78b7e6ba11f76b616ab56c96e2ef4e48970b1b253e65b8fd2af7acf210bc700`. The base side is Cindy's reconstruction, not a historical base-TRP corpus. Additional Markdown prose fixtures provide the ordinary-instruction baseline; SHA-256: `7561444c00cec52af426c46e81c454068e1fe00292a0ef0e9946cf82993d806d`. They were authored and frozen separately after the original comparison.

Hashes identify the inspected sources without requiring disclosure of the private preset. This paper is a standalone design account; reproducing runtime results would require the implementation, compatible host configuration, and an executed evaluation protocol.

TeleWire TRP-AO · November 3, 2026
Method and implementation: Ryah · Drafting and token analysis: Cindy
Reproducible token measurements; controlled behavioral evaluation proposed.